

blueprint_analysis 依赖包

横纵分析法深度研究报告

研究时间：2026-06-04 | 所属领域：前端工程工具链 | 研究对象类型：项目依赖包组合

作者: 数字生命卡兹克

研究时间：2026-06-04 | 所属领域：前端工程工具链 | 研究对象类型：项目依赖包组合

一、一句话定义

`blueprint_analysis` 当前的依赖组合，是一套围绕 `Vue 3 + Vite 8 + Vite+ + Vitest + Oxlint + Stylelint` 搭出来的现代前端工具链：运行时很轻，构建和测试很新，静态检查正在从 ESLint 时代切到 OXC/Rolldown 时代。

它的风险也正好来自这里。

不是依赖太多，而是核心工具链太靠近前沿版本。前沿版本带来速度，也带来 peer dependency、registry 发布节奏、Node 版本约束和 CI 可复现性的摩擦。

二、当前依赖画像

1. 顶层依赖

生产依赖只有两个：

包	当前安装	npm latest	角色	判断
<code>vue</code>	<code>3.5.30</code>	<code>3.5.35</code>	SPA 运行时	可做 patch 升级
<code>qrcode</code>	<code>1.5.4</code>	<code>1.5.4</code>	分享码登录二维码	稳定，继续保留

开发依赖是主战场：

包	当前安装	npm latest / 状态	角色	判断
<code>vite</code>	<code>8.0.16</code>	<code>8.0.16</code>	构建与 dev server	当前最新，保持
<code>vite-plus</code>	<code>0.1.23</code>	<code>0.1.24</code>	统一 CLI / Vite+ 入口	暂时保持 0.1.23
<code>@vitejs/plugin-vue</code>	<code>6.0.7</code>	<code>6.0.7</code>	Vue SFC 支持	当前最新，保持

包	当前安装	npm latest / 状态	角色	判断
vitest	4.1.7	registry latest 为 4.1.7	单元测试	当前可用线，保持
@vitest/coverage-v8	4.1.7	registry latest 为 4.1.7	覆盖率	必须和 Vitest 锁同版本
typescript	6.0.3	6.0.3	类型系统	当前最新，保持
vue-tsc	3.3.3	3.3.3	Vue 类型检查	当前最新，保持
oxlint	1.68.0	1.68.0	JS/TS lint	当前最新，保持
stylelint	17.12.0	17.12.0	CSS/Vue style lint	当前最新，保持
stylelint-scss	7.0.0	7.2.0	SCSS/CSS 扩展规则	可评估升级
prettier	3.8.3	3.8.3	格式化	当前最新，保持
@types/node	25.9.1	25.9.1	Node 类型	建议考虑对齐 CI Node 24

2. 本地依赖树异常

npm ls --depth=0 --json 显示两个 extraneous 包：

包	版本	含义
@emnapi/wasi-threads	1.2.1	存在于 node_modules ，但不在当前 lockfile 依赖关系里
tslib	2.8.1	同上

这通常不是 package.json 的结构风险，而是本地 node_modules 经过多轮安装、超时、版本切换后留下的物理残留。真正提交前应通过一次干净安装验证：

```
npm ci
npm run test:ci
```

如果本地 `npm ci` 因已有目录残留不干净，直接删除 `node_modules` 后重装即可。这个动作不应该改变源码，只是清理安装状态。

3. 安全审计

`npm audit --json` 当前给出 1 个 high:

漏洞包	来源链路	严重级别	修复方式
<code>fast-uri@3.1.0</code>	<code>stylelint -> table -> ajv -> fast-uri</code>	high	<code>npm audit fix</code> 可把它升到 <code>3.1.2</code>

`npm audit fix --dry-run --json` 显示修复只会变更一个间接包:

`fast-uri 3.1.0 => 3.1.2`

这是一种很干净的安全修复，不需要大版本升级，也不需要替换 Stylelint。我的建议是单独提交这个 lockfile 修复，并跑完整验证。

三、纵向分析：这个依赖组合是怎么长出来的

1. 第一阶段：轻量 Vue SPA

这个项目的底色不是大型业务系统，而是一个蓝图分析工具。它的生产依赖只有 `vue` 和 `qrcode`，说明运行时非常克制。核心逻辑主要在本地 JSON 解析、布局计算、图片资源映射、分享码查询这些路径上，前端框架本身没有承担复杂状态管理。

这也是为什么项目没有必要引入 Next.js、Nuxt、React Server Components 或大型状态库。它不是内容站，不需要服务端渲染来抢首屏 SEO；它也不是多团队中后台，不需要过早引入路由级模块化和全局状态治理。

早期选择 `Vue + Vite` 是合理的。Vue 负责界面响应式，Vite 负责快速开发和构建，Vitest 接住工具函数和 composable 的测试。整个组合有一种典型的小型专业工具气质：少依赖，少框架仪式感，把工程复杂度留给蓝图领域逻辑，而不是留给应用壳。

2. 第二阶段：质量门禁开始成形

后来项目加入了 TypeScript、Vitest、Stylelint、Prettier、i18n 检查脚本。这一阶段的重点不是「跑得更快」，而是「不要在蓝图解析这种细碎规则里悄悄坏掉」。

`vitest` 在这里很自然。它和 Vite 的转换管线同源，测试 Vue/TS 工具函数的成本低。项目现有测试已经覆盖：

- 蓝图解析和摘要生成
- 路径拓扑和布局
- 分享码查询 session / cookie / QR 流程
- i18n message 完整性

这套测试不是很大，但方向对。对这种工具型项目来说，比起 E2E 大而全，更有价值的是把纯逻辑和用户可感知的异步流程守住。

3. 第三阶段：Vite 8、Rolldown、OXC 进入主线

这次迁移把项目推到了新的工程周期：`vite@8.0.16`、`typescript@6.0.3`、`oxlint@1.68.0`、`vite-plus@0.1.23`。

这个变化背后有两条线。

一条是 Vite 自己的线。Vite 8 进入 Rolldown 版本线，底层构建和转换能力向 Rust/OXC 生态靠近。项目从 Vite 6 跳到 Vite 8，不只是版本号变了，而是把未来几年 Vite 工具链的方向提前接进来了。

另一条是 lint 的线。ESLint 很强，但它的强大来自规则生态、插件生态和 JavaScript 运行时的灵活性。Oxlint 的思路不同，它用 OXC 的速度和基础规则覆盖常见 JS/TS 问题，把「快」作为第一性。这个项目当前保留 `vue-tsc` 和 `stylelint`，用 Oxlint 替换 ESLint，是正确拆分：Oxlint 管 JS/TS 基础问题，Vue 模板和类型交给 `vue-tsc`，样式交给 Stylelint。

这不是「少装一个 ESLint」这么简单，而是把检查责任重新分配。

4. 第四阶段：Vite+ 作为统一入口，但还不能完全放手

`vite-plus` 的引入，是为了把 `dev/build/preview` 这些入口统一到 `vp`。但现在的 Vite+ 仍处在早期版本段，最典型的证据就是 `vite-plus@0.1.24`。

npm registry 上 `vite-plus` latest 是 `0.1.24`，但它依赖的 `@voidzero-dev/vite-plus-test@0.1.24` 要求 peer：

```
{
  "@vitest/coverage-v8": "4.1.8",
  "@vitest/ui": "4.1.8",
  "@vitest/coverage-istanbul": "4.1.8"
}
```

而 `vitest@4.1.8` 和 `@vitest/coverage-v8@4.1.8` 当前在 npm registry 查询不到。也就是说，Vite+ 的最新小版本跑在了一个尚未完全发布的 Vitest 版本面上。

这就是依赖包分析里最重要的纵向判断：项目可以拥抱 Vite+，但不能盲目追 Vite+ latest。现在 `pin vite-plus@0.1.23` 是工程上更稳的选择。

四、横向分析：同类方案怎么比

1. 构建工具：Vite 8 vs Webpack / Rspack / Parcel

当前项目选择 Vite 8，是一个偏前沿但合理的选择。

Webpack 的优势是成熟、插件多、企业存量。缺点也明显：对这种 Vue 单页工具来说，Webpack 的复杂度收益不高。项目没有复杂多入口、Module Federation、历史 loader 生态包袱，硬上 Webpack 只是把工具链变重。

Rspack 是另一个强选项。它的速度和 Webpack 兼容性都不错，适合「Webpack 存量很大但想加速」的团队。但本项目没有 Webpack 存量，直接走 Vite 更自然。

Parcel 的定位是零配置和自动化，但 Vue/Vitest/TypeScript 的组合生态没有 Vite 贴合。蓝图项目需要的是可控的 alias、base path、proxy、coverage、Vue SFC 类型检查。Vite 在这些点上更顺。

所以横向看，Vite 8 的位置很清楚：它不是最保守的，但它是这个项目在「轻量、现代、Vue 生态贴合」三者之间的最佳平衡。

2. 统一工具入口：Vite+ vs 原生 npm scripts / Turborepo / Nx

`vite-plus` 的竞争对象不是 Vite，而是散落的工程脚本。

原生 npm scripts 的好处是透明，每一条命令都看得见。缺点是项目一复杂，脚本会变成一串 `&&`。当前项目已经有 `lint`、`stylelint`、`il8n`、`coverage`、`build`、`version:bump`。如果继续扩展，入口会越来越散。

Turborepo 和 Nx 更适合 monorepo。它们的缓存、任务图、包间依赖，在单包 Vue 工具里有点重。这个项目目前不需要 workspace 级任务调度。

Vite+ 的生态位刚好在中间：比纯 npm scripts 有统一工具入口，比 Nx/Turbo 轻。问题是它还年轻，版本之间和 Vitest/Vite/Oxlint 的发布时间差可能造成 peer mismatch。因此当前策略应该是「用它做入口，但不要让它接管所有事情」。

项目现在让 `dev/build/preview` 走 Vite+，测试仍直接用 Vitest，lint 直接用 Oxlint。这是稳的。

3. Lint：Oxlint vs ESLint vs Biome

ESLint 的强项是规则生态，尤其是 Vue、React、import、accessibility、工程风格这类复杂规则。但它慢，依赖多，配置也越来越重。

Oxlint 的强项是速度。它适合接手 JS/TS 基础正确性规则，例如未使用变量、无效赋值、空文件、一些可静态判断的问题。它的问题是生态还不能完整替代 `eslint-plugin-vue`。

Biome 是另一个横向对手。它同时做 formatter 和 linter，速度也快。但本项目已经保留 Prettier，且这次迁移明确不希望格式化结果大幅变化。引入 Biome 会把 lint 和 format 两条线一起重写，diff 风险比 Oxlint 更大。

所以当前组合最稳：

- `oxlint`：JS/TS 快速基础检查
- `vue-tsc`：Vue SFC 与模板类型检查
- `stylelint`：样式检查
- `prettier`：格式化
- `godana`：额外静态分析信号

这个分工比「用某一个工具包打天下」更适合当前项目。

4. 测试：Vitest vs Jest / Playwright

Jest 成熟，但和 Vite/Vue 现代 ESM 管线并不天然同频。要让 Jest 跑得舒服，需要额外处理转换、alias、ESM、Vue SFC。这些工作对当前项目没有收益。

Playwright 是 E2E 的强项，但它不是单元测试替代品。蓝图项目现在最容易出错的地方是解析、布局、模板映射、session/cookie 异步流程。Vitest 更适合守这些逻辑。

未来如果要补 E2E，应只覆盖少量用户主路径：

- 默认蓝图加载
- 本地 JSON 上传/粘贴解析
- 分享码查询二维码流程
- 主要布局视图渲染

不建议现在把 Playwright 变成必需依赖。它会增加安装体积和 CI 复杂度。

5. Runtime：Vue vs React / Svelte

Vue 继续适合这个项目。

React 的生态更大，但迁移成本没有收益。蓝图项目的交互是表单、面板、SVG 布局、computed 派生状态。Vue 的 Composition API 和 computed/ref 模型很贴这种状态图。

Svelte 更轻，但生态、测试和团队熟悉度未必更好。当前项目已经有 Vue 测试和组件结构，迁移到 Svelte 只是换皮，不解决真正问题。

真正要优化的不是换框架，而是继续拆领域逻辑。比如把蓝图 domain、share query service、layout rendering 的边界进一步清楚化。

五、横纵交汇洞察

1. 历史把项目推向了「轻运行时、重工具链」

这个项目的生产依赖很少，说明运行时没有被框架欲望吞掉。可与此同时，开发依赖很先进：Vite 8、Vite+、TypeScript 6、Oxlint、Vitest 4。

这形成一个有意思的结构：用户下载到浏览器里的东西很克制，开发者本地和 CI 里的工具链很锋利。

我的判断是，这个方向对，但要管理好节奏。依赖包的风险不在 Vue 和 qrcode，而在工具链版本面太新时的 peer dependency 同步问题。`vite-plus@0.1.24` 就是一个活例子。

2. 当前最值得修的不是大版本，而是 lockfile 安全和安装卫生

如果现在问「下一步升级什么」，直觉可能会看 outdated。但真正最高收益的动作不是立刻升 Vue 或 stylelint-scss，而是先处理：

1. `fast-uri` high audit
2. 本地 `node_modules` extraneous 残留
3. `@types/node` 与 CI Node 版本的口径

这些问题不炫，但它们影响可复现性和安全信号。工具链迁移已经够大，下一步应该把地面扫干净。

3. Vite+ 应该继续保守 pin

Vite+ 的价值是统一入口，不是追 latest。

当前 `vite-plus@0.1.23` 能配合 `vitest@4.1.7`、`@vitest/coverage-v8@4.1.7` 跑通；`0.1.24` 要求尚不存在的 4.1.8。这个时候升级 latest 不是现代化，是主动引入不可安装状态。

所以策略应写进团队规则：

Vite+ 只有在 npm view 确认其 peer 版本已经真实发布后再升级。

4. TypeScript 6 是一把提前打开的门

TypeScript 6 已经暴露 `baseUrl` 弃用问题，这次迁移通过移除 `baseUrl` 解决了。这里的信号不是「TS 6 麻烦」，而是项目以后会更早遇到生态里的类型变化。

这要求后续升级时更重视 typecheck，而不是只跑 build。当前 `npm run build` 已经包含：

```
npm run typecheck && npm run vp -- build
```

这是正确姿势。

5. 三个未来剧本

最可能的剧本

项目继续保持 Vue + Vite 8 主线，Vite+ 保守 pin，Oxlint 负责快速 lint，Qodana 作为补充。依赖升级以 patch/minor 为主，每次升级跑 `npm run test:ci`。这是最稳的路线。

最危险的剧本

为了追最新，把 `vite-plus`、`vitest`、`coverage-v8` 一起放宽到 caret，结果某个 peer 版本在 registry 上没完全发布，CI 或新机器安装失败。这个风险已经出现过一次，不能当成理论风险。

最乐观的剧本

Vite+ 稳定下来后，`vp check`、任务缓存、统一 CLI 能进一步减少脚本碎片。项目再把蓝图 domain 和 share query service 的边界整理好，形成一个「轻运行时 + 高速工具链 + 清晰领域模型」的小而硬的工具项目。

六、建议动作清单

立即做

动作	命令	风险	收益
修复 <code>fast-uri</code> audit	<code>npm audit fix</code>	低	消除 1 个 high
清理安装残留	删除 <code>node_modules</code> 后 <code>npm ci</code>	低	去掉 extraneous
继续 pin <code>vite-plus@0.1.23</code>	不升级	低	避免 0.1.24 peer 缺失
保持 Vitest/coverage 同版本	<code>vitest</code> 与 <code>@vitest/coverage-v8</code> 都锁 <code>4.1.7</code>	低	避免 coverage peer mismatch

可以排期

动作	建议
升级 <code>vue</code> 到 <code>3.5.35</code>	patch 更新，跑 <code>npm run test:ci</code>
升级 <code>stylelint-scss</code> 到 <code>7.2.0</code>	先跑 <code>npm run lint:style</code> ，再跑完整 CI

动作	建议
对齐 <code>@types/node</code>	如果 CI 固定 Node 24，可考虑改到 <code>^24.x</code> ，避免 Node 25 API 误用
增加依赖升级说明	在计划文档或 README 里写清 Vite+/Vitest peer 检查规则

暂时不要做

不建议动作	原因
升级 <code>vite-plus@0.1.24</code>	依赖的 Vitest 4.1.8 包当前不可用
引入 Biome 替代 Prettier/Oxlint	会改变格式化和 lint 两条线，迁移面过大
引入 Next.js/Nuxt	当前项目没有 SSR/路由站点需求
立刻引入 Playwright E2E	当前风险主要在纯逻辑和 session 流程，Vitest 更高收益

七、信息来源

- 本地 `package.json`，访问时间：2026-06-04。
- 本地 `npm ls --depth=0 --json`，访问时间：2026-06-04。
- 本地 `npm outdated --json`，访问时间：2026-06-04。
- 本地 `npm audit --json` 与 `npm audit fix --dry-run --json`，访问时间：2026-06-04。
- npm registry: `npm view vite version engines peerDependencies dependencies dist-tags time.modified --json`，访问时间：2026-06-04。包页面：<https://www.npmjs.com/package/vite>
- npm registry: `npm view vite-plus version engines peerDependencies dependencies dist-tags time.modified --json`，访问时间：2026-06-04。包页面：<https://www.npmjs.com/package/vite-plus>
- npm registry: `npm view @voidzero-dev/vite-plus-test@0.1.24 peerDependencies dependencies --json`，访问时间：2026-06-04。包页面：<https://www.npmjs.com/package/@voidzero-dev/vite-plus-test>
- npm registry: `npm view vitest version engines peerDependencies dependencies dist-tags time.modified --json`，访问时间：2026-06-04。包页面：<https://www.npmjs.com/package/vitest>

- npm registry : `npm view @vitest/coverage-v8 version peerDependencies dependencies dist-tags time.modified --json` , 访问时间: 2026-06-04。包页面: <https://www.npmjs.com/package/@vitest/coverage-v8>
- npm registry : `npm view vue version peerDependencies dependencies dist-tags time.modified --json` , 访问时间: 2026-06-04。包页面: <https://www.npmjs.com/package/vue>
- npm registry : `npm view oxlint version engines peerDependencies dist-tags time.modified --json` , 访问时间: 2026-06-04。包页面: <https://www.npmjs.com/package/oxlint>
- npm registry : `npm view stylelint version engines dependencies dist-tags time.modified --json` , 访问时间: 2026-06-04。包页面: <https://www.npmjs.com/package/stylelint>
- npm registry : `npm view stylelint-scss version engines peerDependencies dependencies dist-tags time.modified --json` , 访问时间: 2026-06-04。包页面: <https://www.npmjs.com/package/stylelint-scss>
- npm registry : `npm view typescript version engines dist-tags time.modified --json` , 访问时间: 2026-06-04。包页面: <https://www.npmjs.com/package/typescript>
- GitHub Advisory Database : `fast-uri` path traversal advisory , 访问时间: 2026-06-04。<https://github.com/advisories/GHSA-q3j6-qgpj-74h6>
- GitHub Advisory Database : `fast-uri` host confusion advisory , 访问时间: 2026-06-04。<https://github.com/advisories/GHSA-v39h-62p7-jpjc>

八、方法论说明

本报告使用纵横分析法: 纵向看依赖组合从轻量 Vue SPA 到 Vite 8 / OXC 工具链的演进, 横向比较同类构建、lint、测试和运行时方案, 最后交叉得出升级节奏与风险控制建议。